

基于解码器式 Transformer 架构的 古诗生成器

实验报告

第十四组

何琪铭 丁佳怡

李纳 何馨怡

2026 年 6 月 18 日

复旦大学

课程名称：人工智能导论

指导教师：丁恒辉

目录

1 实验背景：数据和项目结构说明	2
1.1 数据就绪说明	2
1.2 实验环境	3
1.3 项目结构	3
2 解码器式 Transformer 架构	4
2.1 因果掩码与自回归生成机制	5
2.2 缩放点积注意力与梯度稳定性	5
2.3 前馈神经网络 (FFN) 与特征变换	6
3 版本迭代：从写诗到写好诗	6
3.1 数据分类和清洗	7
3.2 主题 embedding：写风格统一的诗	7
3.3 工程优化：训练加速	8
3.4 动态温度策略	9
3.5 生成约束	11
3.5.1 重复惩罚	11
3.5.2 起笔约束	13
3.5.3 押韵约束	13
3.5.4 平仄约束	14
4 参数分析 (结果分析)	15
4.1 模型结构参数 (向量维度 + 中间维度 + 注意力头数 + 层数)	15
4.2 BlockSize	18
4.3 学习率 (Learning Rate) 调参实验	19
4.4 lr 与 batch_size 的协同调参	20
4.5 Dropout	21
附一：写词尝试	22
4.6 v0 版本	22
4.7 v1 版本	22
4.8 v2 版本	23
4.9 反思：数据有限且结构复杂	23
附二：分工	25

概要和报告完备性说明

本实验通过代码实现一个基于解码器式 Transformer 架构的古诗生成器。本组同学补全了完整 Transformer 写诗架构；并在此基础上修改模型、数据训练方式、预测方式，完成多个版本迭代，优化了训练过程和生成诗歌的质量；最后从多个层次实验分析了模型中的超参数，对比不同参数下模型的表现，并试图找到 val-loss 最低的最佳参数。

以下报告首先说明数据和项目结构，然后从解码器式 Transformer 架构，版本迭代优化，参数分析三个方面展开，介绍我们的工作成果。

前端网址：<https://huggingface.co/spaces/HuggingEmily/chinese-poetry-generator>

报告完备性说明：

- 因果掩码的作用，缩放点积除 $\sqrt{d_k}$ 的原因，FFN 的作用已在**解码器式 Transformer 架构**中说明
- temperature 对比实验在**动态温度策略**中展示结果并给出解释
- 过拟合分析在 **Dropout** 中体现，事实上我们还尝试了调节 weight_decay，但因为本身没有严重过拟合，效果不明显，未展示
- loss_epoch 曲线或该曲线关于各个超参数的归纳曲线分布在**参数分析**中

1 实验背景：数据和项目结构说明

1.1 数据就绪说明

实验数据源自 Hugging Fac 镜像源的真实唐宋古诗数据集。为保证样本的代表性并避免诗词种类过于单一，我们首先对全量数据进行了随机打乱，并抽取 100,000 首古诗作为基础样本库。在清洗文本时，删除了无效符号和空行。由于古诗的换行和格式非常重要，脚本严格以首为单位切分诗歌，并在诗歌之间保留换行符，以便模型后续学习诗歌的边界结构。

为了有效评估模型性能并观测过拟合现象，我们以首为单位对数据进行了划分。全量诗歌按 9:1 的比例切分为训练集和验证集，两部分数据相互隔离、不重叠。切分好的文本在编码后分别转为一维张量，并以 PyTorch 二进制文件的格式存为 train_data.pt 和 val_data.pt。

考虑到中国古诗词单字独立成义、讲究对仗的特点，本项目没有采用常规的分词工具，而是直接使用字符级切分。不仅可以大幅缩小词表体积，也能有效避免生僻字导致的未登录词问题。脚本统计了全量文本中所有不重复的字符，最终得到一个包含 8319 个字符的词表。然后我们在代码中建立了字符与数字 ID 的双向映射字典 (stoi 和 itos)，并将词表信息以 JSON 格式保存为 vocab.json(词表大小 $V=8319$)。

1.2 实验环境

本项目采用 GitHub 进行版本管理与团队协作,远程仓库地址为:https://github.com/Jiayi-Ding/PJ9_transformer.git

由于古诗数据量较大 (抽样约 10 万首)，本地 CPU 训练耗时长，我们在本地完成数据准备后，上传数据集到 Kaggle Notebook，使用其免费 GPU 资源 (T4×2) 进行云端训练。

1.3 项目结构

本项目由数据准备、数据流水线、模型定义、配置训练与文本推理五大核心板块构成。项目在根目录下包含五个核心脚本文件。

1. prepare_data.py 负责原始古诗数据集的清洗与冷启动，建立覆盖 8319 个去重字符的词表并保存为 vocab.json，同时将文本编码为 Token ID 序列，按 9: 1 划分保存为二进制张量 train_data.pt 和 val_data.pt
2. dataset.py 继承自 PyTorch 的 Dataset 基类，负责在一维长 Token 序列上以指定窗口长度进行动态滑窗切片，自动构建错位一格的标签序列以满足自回归下一字预测的任务定义
3. model.py 定义纯粹 GPT 式 Decoder-only 架构，内部集成词嵌入层与可学习的绝对位置编码，纵向堆叠了 4 层 TransformerBlock(采用 Pre-LN 拓扑，内含多头因果注意力、前馈神经网络与双重残差连接)，并在最顶层通过 nn.Linear 定义 LLM Head 将特征投射回 8319 类分类空间
4. train.py 整个项目的训练与监控驱动核心，调度命令行超参数与硬件设备，在每轮 Epoch 结束时评测验证集 Loss 并将最优模型权重持久化为 ckpt_best.pt，最终绘制收敛折线图 loss_curve.png

5. predict.py 用于自回归文本生成与交互式测试，加载训练好的权重，引入温度系数调节多分类概率采样，实现古诗的续写。

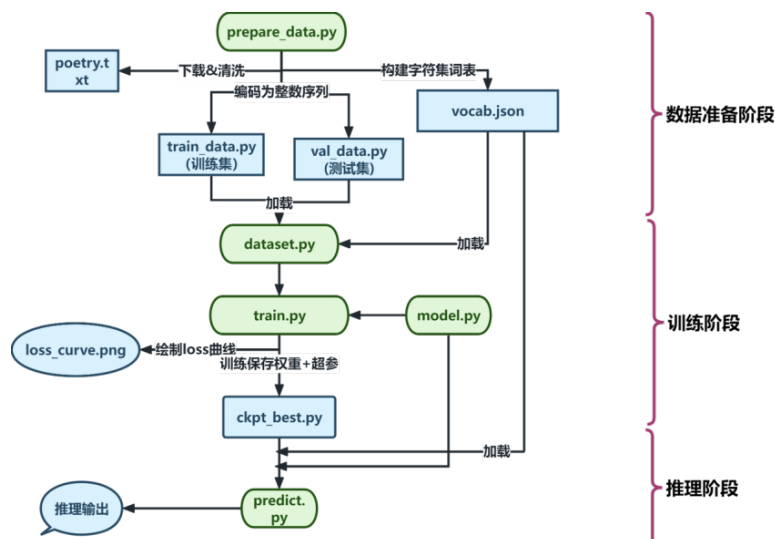


图 1: 代码结构

2 解码器式 Transformer 架构

本文采用字符级 Decoder-only Transformer 构建古诗生成模型，将古诗生成视为一个自回归语言建模任务。模型根据已经出现的字符预测下一个字符，并通过不断重复这一过程完成整首诗的生成。

模型输入为字符编号序列。首先，字符编号经过 Token Embedding 转换为连续向量，同时加入 Position Embedding，使模型能够区分字符在序列中的位置。在启用主题控制时，还会加入 Topic Embedding，并将同一主题向量添加到序列中的每个位置，使模型在生成过程中持续获得山水田园、边塞征戍、思乡怀人等主题信息。三类嵌入相加后，得到维度为 256 的初始字符表示。

模型主体由 4 个相同的 Transformer Block 堆叠而成。每个 Block 包含一个多头因果自注意力子层和一个前馈神经网络子层，并采用 Pre-LN 与残差连接结构。经过 4 个 Transformer Block 后，隐藏表示还会通过一个最终的 LayerNorm，再输入语言模型输出层。输出层将每个

位置的 256 维表示映射到整个字符词表，得到各候选字符的预测分数（见图2）。

以下将从三个方面深入剖析该架构中的核心组件设计，以及它们对古诗生成效果的具体贡献。

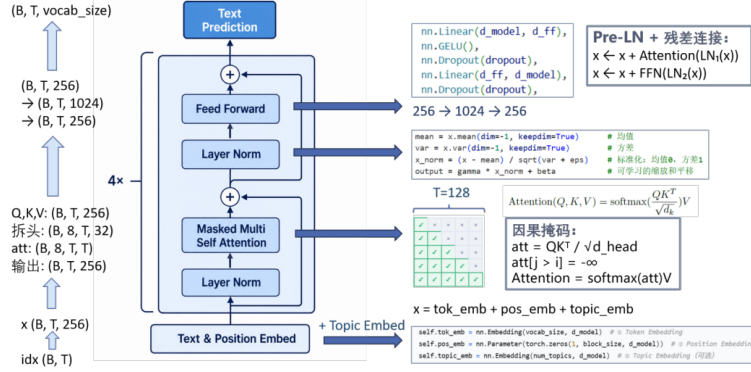


图 2: 解码器式 Transformer 架构

2.1 因果掩码与自回归生成机制

为了保证模型在训练和生成时严格遵循自回归特性，即预测当前字符时只能依赖历史信息，注意力层使用了**因果掩码 (Causal Mask)**。

具体而言，在计算自注意力分数时，对于序列中的第 i 个位置，因果掩码将将其与所有未来位置 ($j > i$) 的注意力分数设为 $-\infty$ 。经过 Softmax 归一化后，这些未来位置的注意力权重变为 0。通过这种方式，模型只能关注到自己以及前面的字符，无法利用未来信息。这不仅确保了训练过程与实际生成过程的一致性，防止了在预测当前词时“偷看”未来词的作弊行为，也是单向语言模型能够正确进行条件概率建模的前提。

2.2 缩放点积注意力与梯度稳定性

在计算自注意力时，模型使用缩放点积注意力公式：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

其中 d_k 是 Key 向量的维度 (本模型中为 32)。引入缩放因子 $\frac{1}{\sqrt{d_k}}$ 是为了平衡点积结果的数值量级。

当维度 d_k 较大时, 点积 QK^T 的结果方差会随之增大, 导致部分元素的绝对值显著大于其他元素。如果直接进行 Softmax 操作, 较大的值会使得 Softmax 函数的输出分布极度陡峭 (趋近于 One-hot 编码), 梯度会变得极小, 影响反向传播的效率, 即导致“梯度消失”问题。通过除以 $\sqrt{d_k}$, 可以将点积结果的方差归一化到约等于 1 的水平, 保证了 Softmax 梯度处于适中且稳定的区域, 从而维持了模型在深层网络中的良好收敛性能。

2.3 前馈神经网络 (FFN) 与特征变换

Transformer Block 中的前馈神经网络 (Feed Forward Network, FFN) 对每个时间位置的字符表示独立进行非线性变换。其设计包含两个全连接层和一个激活函数:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (2)$$

在本模型中, 第一层将维度从 256 扩展到 1024, 第二层再将其从 1024 压缩回 256。

FFN 的作用不仅是对注意力层提取的上下文融合特征进行非线性处理, 更为模型提供了更高维度的特征组合空间。256 维的隐藏空间经过升维至 1024 维后, 模型可以在这个更宽泛的特征空间中捕捉到更复杂、更难分离的字符组合规律 (例如诗句间的律诗对仗或语境关联)。随后降维回 256 维, 相当于对丰富的语义特征进行过滤和提炼。配合 GELU 激活函数, FFN 赋予了模型更强的非线性表达能力, 而内部穿插的 Dropout 层则有效抑制了深层网络可能出现的过拟合现象。

3 版本迭代：从写诗到写好诗

我们在原型模型 (基于解码器式 Transformer 的字符级语言模型) 的基础上, 围绕**数据质量**、**风格可控**、**训练效率**、**推理约束**四个维度进行了多轮迭代, 逐步将生成结果从“长短不一的文字序列”提升为“具有一定格律和意境的古诗”。下面分小节介绍每项改进及其效果。

3.1 数据分类和清洗

原始数据问题：从 Hugging Face 获取的唐宋古诗数据存在缺字 (以 “ ” 标记)、繁简混用、格式杂乱、杂言混杂等问题，直接训练会导致模型学到大量噪声，生成时出现异常字符或结构混乱。为了解决以上问题，我们做出如下改进：

1. **繁体转简体**：使用 OpenCC 将所有文本统一为简体，减少词表冗余，提高字符覆盖率。
2. **无效样本过滤**：剔除缺字、格式异常或过短的无效诗歌，保证训练数据质量：
 - 检测文本中的 (缺字标记) → 丢弃
 - 按换行或标点拆句，若某行中文字符数为 0 → 丢弃
 - 整首诗总中文字符数 < 10 → 丢弃
3. **体裁自动分类**：统计每句的中文字符数：若五言句占比 $\geq 60\%$ 则归为“五言”，七言句 $\geq 60\%$ 则归为“七言”。考虑到五言和七言在格律、节奏上差异显著，我们分别为两类体裁独立训练模型 (通过 `--genre` 指定)，避免了杂言样本的干扰。
4. **类别均衡**：为五言、七言分别补齐至 100,000 条样本，防止因分类后某类体裁数据量不足而造成的训练效果退化。

最终效果：获得了干净、类别明确的数据集，有效清理了 token 中的不规则符号及繁体字，提升了 token 的质量，同时也为后续的格律约束和风格学习奠定了基础。

3.2 主题 embedding：写风格统一的诗

为了避免生成诗歌进入短程有序、长程无序的辞藻堆砌状态，我们决定为整首诗引入“主题”变量来使模型像个真正的诗人一样在整首诗写作过程中保持风格统一，而不仅仅是词和词的连接。技术实现上有两个难点：1. 实现大量诗歌的主题分类；2. 将“主题”标签融入训练中。

为了实现十万首诗歌的主题分类：1. 1. 我们手动维护了一个字典，其中包含十八种风格诗歌，每种风格下记录了该风格的 50 70 个常见字 (例如：“landscape”: [“山”, “水”, “江”, “河”, …… , “岭”, “岳”, “川”])。1.2. 通过每首诗末尾的 “\n\n” 判定诗与诗的边界，构

建 Boundary.json 文件；1.3. 检索字典，比较每首诗中的字在不同主题下出现的频率大小，选择最大频率的主题作为诗歌主题，打上主题标签。如此重复十万次。

在打了主题标签的基础上，为实现将主题标签融入训练：2.1. 类比处理字的思路，我们赋予主题一个 ID，然后映射为一个向量 (topic embedding)，加入原本的 x 中，有

$$x = tok_{emb} + pos_{emb} + topic_{emb}$$

；2.2. 从而主题内容跟随 x 进入 Transformwe Block，训练结束模型就会学习到不同主题的语料，在预测时基于选择的风格 (或者基于字典自动检测的风格) 输出对应风格的诗歌。

3.3 工程优化：训练加速

即使在 kaggleT4×2 的 GPU 环境下进行训练，单轮训练所需时间仍旧达到了十分钟有余，为了缩短迭代周期，加快训练速度，方便后续的版本迭代效果检验及调参工作，我们综合运用了如图所示多种工程优化手段对代码进行处理：

优化方法	优化逻辑
混合精度训练 (AMP)	启用 torch.autocast + GradScaler，以 FP16 执行前向与反向计算，显存占用减半
Dataloader 多进程 + pin_memory	设置 num_workers=4、pin_memory=True，并行加载数据并锁页内存，消除 I/O 瓶颈
AdamW 融合版本	开启 fused=True，将多次核函数调用合并为一次，加速参数更新
矩阵乘法精度优化	调用 torch.set_float32_matmul_precision('high')，利用 Tensor Core 加速 FP32 矩阵乘，矩阵运算提速
torch.compile 模型编译	使用 torch.compile 对模型进行优化 (仅 PyTorch 2.0+ 且 CUDA 有效)

图 3: 工程优化：加速训练

完成以上改进后，以 “python train.py -genre 5 -use_topic -epochs 3 -batch_size 32 -train_num_samples 50000 ” 为例，仅需 2-3 分钟即可完成训练。

3.4 动态温度策略

温度 (Temperature) 通过调整待预测字的概率分布影响最终决策。即，在较大温度下不同备选项的概率差异缩小，选择的随机性增大，诗歌创新性较大；小温度下概率差异增大，模型倾向选择概率值大的选项，随机性降低，生成作品结构稳定但和数据集趋同，缺少创新性。

temperature = 0.5

月落黄香夜，花开竹叶香。山窗一何处，鸟雀一幽栖。
月色生秦出，山声隔驿分。空知沧海上，一卷白云边。
月色随云出，云声带露归。入花开晓色，映水入秋声。

temperature = 0.7

月里何缘对，桃花吼处香。霜流无限尽，俗事不相逢。
月色向西风，风声应未还。昔能同客泪，犹更宿人歌。
月落青山下，潮回一月通。鱼飞故山上，鹤唳一山同。

temperature = 0.9

月满书无客，花开石有梅。平生偏在阙，少日莫知愁。
月月如不到，日日白云归。映叶秋风急，寻人度月飞。
月出云边外，风来刮镜前。蛩声知响塔，松叶助轻庭。

temperature = 1.1

月色莽团溶，朝垣蹙竭筇。吴樯千里上，门望半松扃。
月白三年发，耆夷二月藏。聊知早延隐，又罢旧公私。
月雨花盈令，秋声蕙出痕。池修犁曲出，泉隐茗僧闲。

如上，在采取动态温度策略之前，以“月”字为代表测试了四组不同温度 (0.5,0.7,0.9,1.1) 下的生成效果，发现温度越大诗歌越不工整：繁体字异体字增多，意境奇怪，语句不通；温度太小时表现好了很多，但存在轻微的用词单调问题，例如第一组 (temperature = 0.5) 中”一”字,”香”字多次出现，意境重合度高。

v12 + v13: 押韵 + 平仄
温度做对比实验 (0.7 0.9 1.1) ; 主题自动选择; 韵脚自动

base-temperature = 0.9

13.1. 月明不见一枝梅，谁是春来已未回。秋风吹尽一溪流，犹有江南水上辉。
13. 2. 月落霜寒雨后天，夜来风露滴秋前。千里江山一径开，铜瓶竹屋自萧阡。
13. 3. 月下山头二十余，白云深处见苍梧。风烟漠漠空多雨，日暮江南又一行。长安水部付吴乡，今古无人似故遗。自是春来无限意，不应更得少年香。
13. 4. 月落风流响玉箫，一声空断隔江城。不随明日登临处，未是春来别有程。春色满堤青草绿，秋光连树翠阴成。惊人只见梅花在，只道今年几秦城。

13. 5. 月下天涯路，春生水际山。夜船寒止水，秋雁暮留山。
13. 6. 月下天台上，云间水木飞。人生千金色，海内万年扉。
13. 7. 月上春山路，云间夕烛辉。不堪为客泪，犹有一年挥。二者方来此，于依亦是灰。一雁何所在，五折更踟回。
13. 8. 月色如冰结，天光似玉壶。一枝闲自笑，万里别相呼。故国连千乘，新年到五湖。花开云片白，叶落水干湖。

base-temperature = 0.7 (以五言为例)

13.9. 月落江城暮，风高塞雁飞。归心方自远，去雁已相飞。
13.10. 月色明窗户，风光满院除。清吟谁与共，白鬓自相储。
13.11. 月落江村夜，风生草木秋。不知闲处士，犹有故人游。去国成都会，开门得暂方。一杯聊自乐，万事未应忘。
13.12. 月落天涯路，风生草木秋。不知春色晚，犹有故人愁。自觉心无事，谁怜鬓已头。伤心千万里，泪断一肠流。

base-temperature = 1.1 (以五言为例)

13.13. 月下临江路，人间落照迟。长安知几日，谁是问归迟。幽寺桐庐好，园林竹径池。风流清浅水，雨洒绿深堤。
13.14. 月下多清景，亭高独倚栏。残云低树影，疏雨湿苔闾。病觉身如梦，闲知世事忙。何时一杯酒，相对醉沧澜。
13.15. 月影江边白，风声塞外迟。故存吾子泪，别作故人痴。
13.16. 月落山头白，风生水底寒。江湖何有约，世事故人输。

图 4: 动态温度策略下不同 base_temperature 的对比

为了保证工整的同时不显陈词滥调，我们提出双层动态温度策略，兼顾稳定和创新优点，具体如下：

第一层：基于熵的动态温度调整：新增一个熵计算函数在每次预测下一个字时评估此时模型的自信程度 (即不同备选项的改成分布差异大小)，概率差异越大，说明模型越自信，输出越保守。此时通过一个基于熵判断的分段函数增大温度，使输出更具创新性。反之相反，熵越小，模型越不自信，则通过分段函数调大温度，通过保守策略使输出风格更稳定。

```
# ===== 自适应温度：先算熵 =====
if use_adaptive:
    # 计算当前概率分布（用基础温度）
    probs = F.softmax(last_logits / max(base_temp, 1e-6), dim=-1)

    # 计算熵:  $-\sum p \cdot \log(p)$ 
    log_probs = torch.log(probs + 1e-8) # 加极小值防止log(0)
    entropy = -(probs * log_probs).sum(dim=-1) # [B]

    # 根据熵调整温度
    # 熵高（犹豫）-> 降温，帮模型做决定
    # 熵低（自信）-> 升温，增加随机性
    if entropy.item() > 2.5: # 很犹豫
        temp = base_temp * 0.5
    elif entropy.item() > 1.5: # 中等犹豫
        temp = base_temp * 0.8
    elif entropy.item() < 0.8: # 很自信
        temp = base_temp * 1.5
    else: # 正常
        temp = base_temp
else:
    temp = temperature
```

图 5: 基于熵的动态温度

```
for _ in range(max_new):
    # 体裁/首颌颈尾的温度调整
    temp_factor = 1.0
    if target_lines == 8:
        if line_count in [2, 3]:
            temp_factor = 0.8
        elif line_count in [4, 5]:
            temp_factor = 1.1
    elif target_lines == 4:
        if line_count in [0, 1]:
            temp_factor = 0.8
        elif line_count in [2, 3]:
            temp_factor = 1.1
```

图 6: 基于位置的动态温度

第二层：基于位置的动态温度调整中国古诗中律诗讲求“首颌颈尾”遵循“起承转合”；绝句讲求前两句对仗工整，后两句略奔放。鉴于此，我们首先写了一个通过标点符号计算句数的位置探测器，然后让律诗颌联所有字温度降低 ($\times 0.8$)，实现对仗工整；颈联温度升高 ($\times 1.1$) 实现风格转折。绝句同理。效果很好。

图4展示了加入动态温度策略后写的诗歌，效果显然比加入动态温度前好了很多。同时，在动态温度下我们也做了不同 `base_temperature`(0.7,0.9,1.1) 的对照，发现此时组与组间的差距缩小很多，这是因为真正的 `temperature` 是 `base_tem` 线性变换后的结果，经处理后的温度都稳定在几个较小的值，超过 1 的字很少（如图7），这说明我们动态温度策略不仅优化了诗歌写作，还降低了 `base_tem` 敏感度，减小了调参难度。

3.5 生成约束

3.5.1 重复惩罚

为了减少古诗生成过程中产生的汉字重复出现的问题，我们对记录已生成 token t 的出现次数 $\text{count}[t]$ ，并按照出现次数，在预测时对其概率 $\text{logits}[t]$ 按照公式：

$$\text{logits}[t]' = \frac{\text{logits}[t]}{1 + (\text{rep_penalty} - 1) \times \text{count}[t]}$$

进行处理（其中 `rep_penalty` 是惩罚系数，默认 1.2），从而有效抑制单字重复的问题。同

熵=5.79, 温度=0.40
熵=4.67, 温度=0.40
熵=5.12, 温度=0.40
熵=4.39, 温度=0.40
熵=3.98, 温度=0.40
熵=0.00, 温度=1.20
熵=3.48, 温度=0.40
熵=4.34, 温度=0.40
熵=3.65, 温度=0.40
熵=3.96, 温度=0.40
熵=3.24, 温度=0.40
熵=0.00, 温度=1.20
熵=5.75, 温度=0.40
熵=3.35, 温度=0.40
熵=5.49, 温度=0.40
熵=4.25, 温度=0.40
熵=4.15, 温度=0.40
熵=0.00, 温度=1.20
熵=4.69, 温度=0.40
熵=1.87, 温度=0.64
熵=4.29, 温度=0.40
熵=3.34, 温度=0.40
熵=3.60, 温度=0.40
熵=0.00, 温度=1.20
熵=5.60, 温度=0.40
熵=1.08, 温度=0.80
熵=4.94, 温度=0.40
熵=4.45, 温度=0.40
熵=3.60, 温度=0.40
熵=0.00, 温度=1.20
熵=5.36, 温度=0.40
熵=4.02, 温度=0.40
熵=4.84, 温度=0.40
熵=4.22, 温度=0.40
熵=4.40, 温度=0.40
熵=0.00, 温度=1.20
熵=0.00, 温度=1.20

=====

月出山林外，山幽树木间。水清天地静，水静月山闲。归去无人去，归来有客来。白云无处处，不是到山飞。

=====

图 7: 变换后的 temperature

时让惩罚过程有意识地跳过换行符和标点，确保输出格式正确不受影响。

体裁：5，起笔字：春

=====

春风吹不尽，春色满寒山。

何处一般人，清风满碧波。闲看花未老，醉伴酒还归。

一日天涯远，三年岁岁迟。一杯多一醉，一醉是三年。

图 8: 重复惩罚前

体裁：5，起笔字：春

=====

春风吹晓色，长日见寒阴。山鸟游仙境，山禽隔野人。

一片雪中秋，人生不可攀。三年无此去，万事不能归。

图 9: 重复惩罚后

3.5.2 起笔约束

自回归生成时，模型无法准确判断所在的位置，常将其误判为句中或句尾，导致输出结构混乱。为了解决这一问题，在编码起笔字前，我们强制插入一个换行符 token，明确标记诗歌起始边界。该约束使模型从新句首开始生成，输出格式整齐，字数计数准确，为后续格律约束提供了可靠的定位基础。



图 10: 起笔约束

3.5.3 押韵约束

传统古诗讲究句末押韵，但语言模型仅凭数据统计难以稳定遵守。我们同样在推理阶段手工施加押韵约束，强制指定位置的字必须来自同一韵部。功能分两步实现：

1. **构建韵母映射表**：基于训练集词表，利用 pypinyin 提取每个汉字的拼音韵母（去除声调数字），保存为 rhyme_dict.json。同时过滤出友好韵母（该韵母下汉字数 30），避免韵字过少导致生成卡死。
2. **推理时动态约束**：用户可通过 -rhyme 指定韵脚（如 ang），或通过 -auto_rhyme 让模型根据第一个偶句末字自动确定韵脚。生成时，在偶数句末字（即押韵位置）通过 rhyme_helper.get_rhyme_group(vc

获取同韵字列表，传递给 `sample_with_allowed`，强制模型只从该候选集中采样。其他位置正常生成，不受押韵约束。

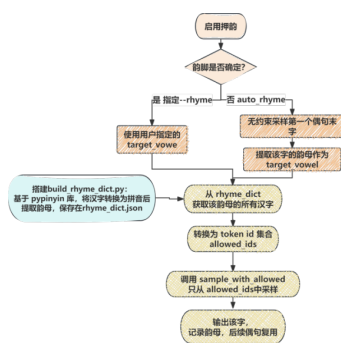


图 11: 押韵结构示意图

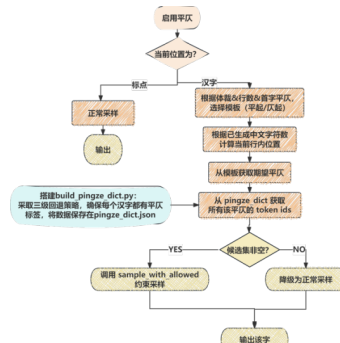


图 12: 平仄结构示意图

3.5.4 平仄约束

古典诗歌 (尤其是近体诗) 对平仄有严格讲究，但纯数据驱动的语言模型难以自动习得。我们决定在推理阶段手工添加平仄约束，即强制模型输出的每一字符符合预设平仄模板。功能实现分两步完成：

1. 构建平仄映射表：基于训练集词表 (vocab.json)，为每个汉字标注平仄 (0= 平, 1= 仄, 2= 未知)。采用三级回退策略进行分类保障覆盖率： 优先使用平水韵库 (pingshui_rhyme) 中古音分类； 简体字未识别则转为繁体再尝试； 仍失败则用拼音法 (普通话声调：一二声为平，三四声为仄) 兜底。最终生成 pingze_dict.json。
2. 推理时动态约束：根据五言/七言、绝句/律诗 (共 8 种模板) 以及起笔字的实际平仄，自动选择平起或仄起格式。生成时，在需要约束的汉字位置计算期望平仄，通过 `pingze_helper.get_tokens_by_ping` 获取所有符合该平仄的 token，传递给采样函数 `sample_with_allowed`，强制模型只从候选集中选取。

考虑到押韵与平仄可能同时启用。当押韵位置 (偶数句末字) 与平仄约束重叠时，先取候选集交集；若交集为空，则优先保证押韵 (放弃该位置平仄约束)，防止生成卡死。其他位置独立施加平仄约束，确保整体格律准确。经此流程，生成的诗句在平仄上基本符合古典范式。

4 参数分析 (结果分析)

4.1 模型结构参数 (向量维度 + 中间维度 + 注意力头数 + 层数)

实验组	固定设置	参数取值	训练轮次
基准参数	block_size=128, batch_size=32, lr=3e-4, train_num_samples=15000 0, seed=42	d_model=256, n_head=8, n_layer=4, d_ff=1024	/
主实验: 固定 d_k 宽度扩展	d_k=32, d_ff=4d_model, n_layer=4, dropout=0.15, early_stop_patience=4	d_model/n_head/d_ff: 192/6/768; 256/8/1024; 320/10/1280; 512/16/2048	最多 30 轮; 采用早停
补充实验: 单变量敏感性 (控制变量)	baseline=256/8/4/1024, block_size=128, dropout=0.1	d_model=192/320; n_head=4/16; n_layer=3/5; d_ff=768/1280	固定 6 轮

图 13: 结构参数实验设置

本组实验主要考察 Transformer 结构容量对生成模型验证集表现的影响, 涉及向量维度 d_{model} 、前馈网络中间维度 d_{ff} 、注意力头数 n_{head} 和层数 n_{layer} 四类结构参数。基础模型设置为 $d_{\text{model}}=256$, $d_{\text{ff}}=1024$, $n_{\text{head}}=8$, $n_{\text{layer}}=4$, 其余训练条件保持一致: $\text{block_size}=128$ 、 $\text{batch_size}=32$ 、 $\text{lr}=3\text{e-}4$ 、 $\text{train_num_samples}=150000$ 、 $\text{seed}=42$ 。其中 d_{model} 控制 token 表示向量维度, d_{ff} 控制 Transformer block 中前馈网络容量, n_{head} 控制多头注意力的并行子空间数量, n_{layer} 控制模型深度。

为清晰比较模型结构参数的影响, 实验分为两组: 第一组为单变量敏感性实验, 每次只改变一个结构参数, 训练 6 轮; 第二组为宽度同步扩展实验, 固定每个注意力头维度 $d_k=32$, 并令 $d_{\text{ff}}=4*d_{\text{model}}$, 使用最多 30 轮训练并加入早停机制。第一组实验结果如下:

从单变量实验可以看出, 五言任务对模型容量的需求相对较低。将层数从 4 层减少到 3 层后, 五言验证 loss 从 3.9800 降至 3.9638, 是五言组中的最好结果; 而单纯增大 d_{model} 或

实验配置	d_model	d_ff	n_head	n_layer	五言 Best Val Loss	七言 Best Val Loss	训练轮次
baseline	256	1024	8	4	3.9800	4.1061	6
调小 d_model	192	1024	8	4	3.9949	4.1618	6
调大 d_model	320	1024	8	4	4.0002	4.0858	6
调小 d_ff	256	768	8	4	3.9881	4.0559	6
调大 d_ff	256	1280	8	4	3.9696	4.0389	6
调少 n_head	256	1024	4	4	4.0008	4.1134	6
调多 n_head	256	1024	16	4	4.0099	4.1000	6
调少 n_layer	256	1024	8	3	3.9638	4.1303	6
调多 n_layer	256	1024	8	5	3.9813	4.0458	6

图 14: 结构参数第一组实验数据

增加注意力头数并没有带来收益,反而使验证 loss 上升。这说明五言数据结构相对短小,基础模型已经具备足够表达能力,继续扩大模型更容易增强对训练集细节的拟合,而不一定提升泛化效果。

七言任务的趋势不同。七言序列更长,对上下文建模和特征变换能力要求更高,因此增大 d_ff 和增加层数均能带来明显改善。其中 d_ff=1280 时七言验证 loss 从 baseline 的 4.1061 降至 4.0389,是单变量实验中的最优结果; n_layer=5 也将验证 loss 降至 4.0458。相比之下,单独调整 n_head 的收益较弱,说明当前 n_head=8、每头维度约为 32 的设置已经较为均衡。

进一步地,为避免单独增大 d_model 时破坏每个注意力头维度,本实验又固定 d_k=32,使 d_model/n_head 保持一致,并同步设置 d_ff=4*d_model。结果如下:

同步扩展实验进一步说明,模型并不是越大越好。五言任务在 d_model=192, d_ff=768, n_head=6 时取得最低验证 loss 3.9663,说明较小模型已经足够;当扩展到 d_model=512 后,五言验证 loss 上升到 4.0684,并且最优 epoch 提前到第 4 轮,表现出更明显的过拟合倾向。七言任务则在 d_model=320, d_ff=1280, n_head=10 时取得最低验证 loss 4.0003,说明适度扩大模型容量可以提升七言建模能力,但继续扩大到 d_model=512 后验证 loss 反而变差。

综合两组实验,结构参数的选择应根据五言和七言任务分别权衡。若优先考虑五言生成,

d_model	d_ff	n_head	n_layer	五言 Best Val Loss	五言 Best Epoch	七言 Best Val Loss	七言 Best Epoch	训练设置
192	768	6	4	3.9663	21	4.0794	29	最多 30 轮, 早停
256	1024	8	4	4.0004	17	4.0181	29	最多 30 轮, 早停
320	1280	10	4	4.0177	9	4.0003	30	最多 30 轮, 早停
512	2048	16	4	4.0684	4	4.0847	6	最多 30 轮, 早停

图 15: dff 协同调节

较浅或较小的模型更合适, 例如 $d_model=256$, $d_ff=1024$, $n_head=8$, $n_layer=3$; 若优先考虑七言生成, 适当增大前馈网络宽度或整体宽度更有效, 例如 $d_model=320$, $d_ff=1280$, $n_head=10$, $n_layer=4$ 。如果希望使用一套相对均衡的结构同时兼顾五言和七言, $d_model=256$ 320、 $d_ff=1024$ 1280、 $n_head=8$ 10、 $n_layer=4$ 是更稳妥的范围。

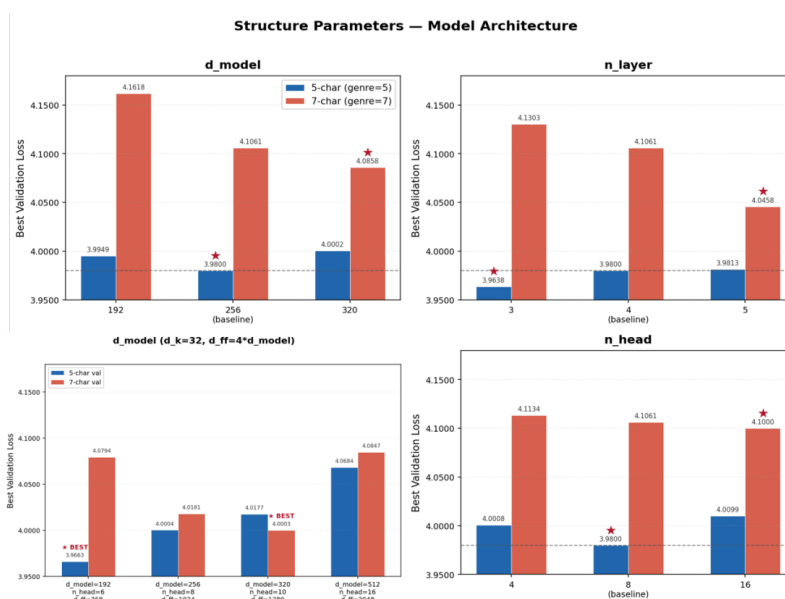


图 16: 结构参数实验结果

由此可见, 在中小规模字符级生成任务中, 结构扩展的关键并不是单纯增加参数量, 而是新增参数能否形成与任务复杂度相匹配的有效容量。五言任务中, 额外宽度更多表现为冗余容量, 导致验证收益下降; 七言任务中, 中等宽度扩展能够补足模型对长上下文和复杂句式的表

达能力，但边际收益迅速递减。由此可见，Decoder-only Transformer 的结构扩展存在任务相关的容量阈值：低于该阈值时，新增参数改善表达能力；超过该阈值后，新增参数更可能放大训练拟合而非提升泛化表现。

4.2 BlockSize

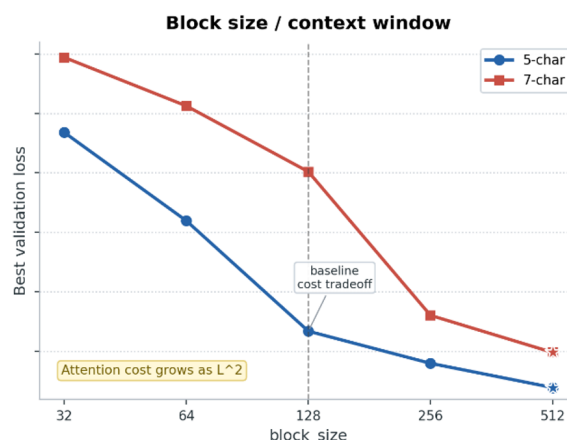


图 17: blocksize 实验结果

发现，随着 `block_size` 的逐渐增大，验证集损失和训练集损失的收敛下限均呈现出先降低后趋于平稳的倒 U 型演进趋势。具体表现为：当 `block_size` 从 32 增大到 128 时，五言的最低 val loss 从 4.10 显著下探至 4.03 附近，七言从 4.41 下探至 4.31 附近。然而，当 `block_size` 进一步拓宽至 256 甚至 512 时，两条损失曲线的下限值几乎不再发生明显位移。此外，在所有空间跨度下，模型直到第 6 个 Epoch 结束依然表现出平稳收敛特征，未触发过拟合抬头。

初步结论：增大 `block_size` 能直接拓宽自注意力机制的时序感受野，赋予模型更强的长程上下文建模能力，这对于捕获古典诗歌内部跨句的押韵、对仗以及整体章法结构具有关键性的助益（这解释了小窗口如 32、64 时损失普遍偏高、无法彻底收敛的现象）。但当 `block_size` 超越特定临界点后，由于单首诗歌的实际 Token 长度本身有限，更长的语义窗口无法提供更多增量信息，增益表现出明显的边际效应递减，反而会线性增加非必要的计算显存负担。

4.3 学习率 (Learning Rate) 调参实验

固定其他超参数不变，取 $\text{epoch} = 20$ ，单独考察学习率策略的影响。对比两组策略：固定学习率 (Fixed LR) 与线性预热 + 余弦退火 (Warmup + Cosine Annealing)，各自取 max_lr 为 $1\text{e-}4$ 、 $3\text{e-}4$ 、 $5\text{e-}4$ 、 $1\text{e-}3$ 进行训练，记录最佳验证损失 (val-loss)，结果如下：

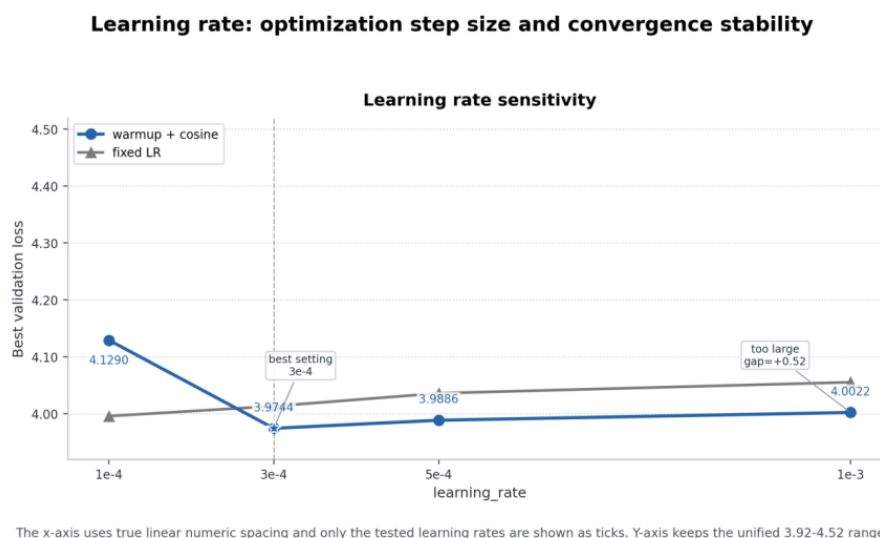


图 18: 学习率两种策略比较

固定学习率分析：随着学习率从 $1\text{e-}4$ 提升至 $1\text{e-}3$ ，最佳验证损失单调上升 ($3.9959 \rightarrow 4.0553$)。小学习率 ($1\text{e-}4$) 下模型虽然验证损失低，且在 20 个 epoch 后仍有良好潜力，但收敛过慢；大学习率 ($1\text{e-}3$) 虽收敛速度上升，但下梯度更新步长过大，模型在损失函数最小值附近来回震荡，难以落入最优谷底，最终性能显著下降。

预热 + 余弦退火分析：新版策略在不同 max_lr 下均表现更优。当 $\text{max_lr}=3\text{e-}4$ 时取得全场最佳验证损失 3.9744，兼顾效率与性能。同时，即使 max_lr 提升至 $1\text{e-}3$ ，验证损失也仅为 4.0022，远优于固定 LR 同等量级下的 4.0553，证明退火机制有效抑制了震荡。

机理分析： fix_lr 在大学习率收敛快但后期震荡，小学习率收敛稳但速度慢。线性预热策略在训练初期梯度方向不稳定时以小步长起步，待方向稳定后再逐步增大至目标学习率，实现快速收敛；余弦退火阶段则逐步降低学习率，在损失函数最小值附近精细搜索，避免跨过最优区域。该策略结合兼顾了收敛速度与最终精度，有效提升了模型性能。

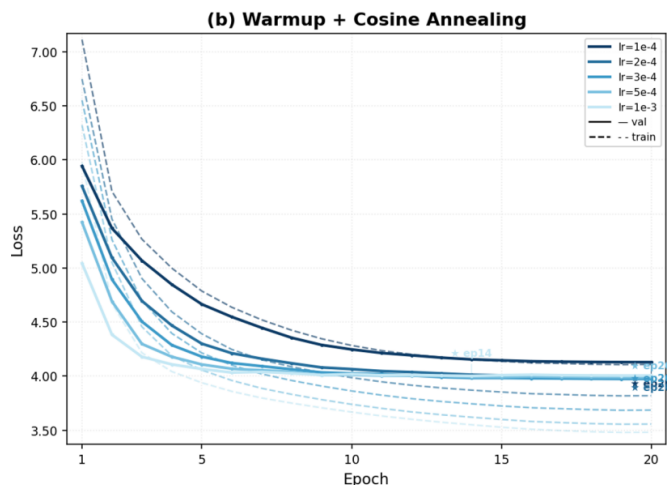


图 19: 不同学习率收敛速度

4.4 lr 与 batch_size 的协同调参

在确定使用线性预热 + 余弦退火调度策略后, 我们进一步考察最大学习率 ($\max_lr$) 与批量大小 (batch_size) 的耦合关系。保持其他超参数不变 ($d_model=256, n_head=8, n_layer=4, \text{block_size}=128, \text{epochs}=15$), 分别取 $\max_lr$ 3e-4, 5e-4, 1e-3 和 batch_size 20, 32, 64 构成 3×3 实验组, 记录各组合的最佳验证损失, 以 lr 为横轴、 val_loss 为纵轴绘制曲线, 可清晰观察到三条随 batch_size 变化的趋势线:

结果分析:

横向 (固定 batch size, 增大 lr): 当 $\text{batch_size}=20$ 时, 最优学习率为 5e-4 ($\text{val_loss}=3.9728$), 1e-3 时性能略有下降; 当 $\text{batch_size}=32$ 时, 3e-4 与 5e-4 表现接近, 最优为 3e-4; 当 $\text{batch_size}=64$ 时, 3e-4 显著优于 5e-4 和 1e-3。整体呈现小 batch 适用中等偏大 lr, 大 batch 适用较小 lr 的趋势。

纵向 (固定 lr, 增大 batch size): $lr=3e-4$ 时, $\text{batch_size}=32$ 最优, 64 反而变差 (梯度估计虽稳定但收敛步数减少, 且更大 batch 可能使模型陷入尖锐极小值); $lr=5e-4$ 时, $\text{batch_size}=20$ 最优, 随着 batch 增大性能下降; $lr=1e-3$ 时, $\text{batch_size}=20$ 与 32 接近, 64 略差。

机理分析: 批次大小影响梯度估计的方差。小 batch (如 20) 梯度噪声较大, 需要较高学习率以“冲破”噪声、保持有效更新; 大 batch (如 64) 梯度估计更精确, 较小学习率即可稳定收

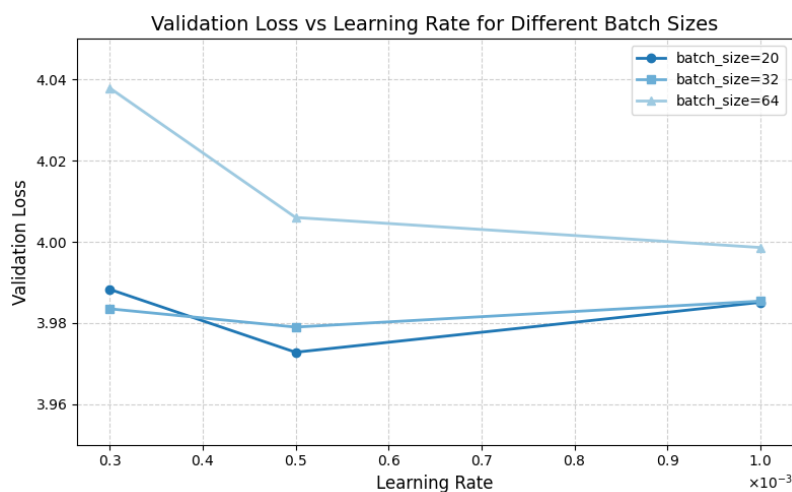


图 20: 学习率与 BatchSize 耦合

敛。然而本实验中 `batch_size` 的边际收益在 64 时减弱，可能因单个 epoch 内更新次数减少所致。

结合实验结果，最优配置为 `lr=5e-4`，`batch_size=20`，在该组合下取得全场最佳验证损失 3.9728。上述规律印证了学习率与批次大小之间存在耦合关系：批次越大，梯度越稳定，可耐受的学习率范围越窄；批次越小，需适当提高学习率以维持更新步长。在实践中需协同调整二者，而非孤立调参。

4.5 Dropout

dropout 通过训练时随机关闭部分神经元增加噪声来避免过拟合，同时 dropout 过大容易发生欠拟合导致 `val_loss` 也增大。我们控制变量（其余参数使用基线参数），分别对五言诗和七言诗调试了四组：`dropout = 0.05, 0.10, 0.15, 0.20`，结果如图（以七言诗为例）：

发现，随着 dropout 增长，`trainloss` 最小值逐步抬升，`trainloss` 和 `valloss` 的交点前移，这证明了过拟合趋势被有效抑制；同时，在 `dropout = 0.20` 时，发现 `valloss` 明显增长，我们认为这是噪声过度增加导致了欠拟合。结合试验，dropout 取 0.1 或 0.15 时效果最好（值得注意，dropout 实际和模型大小相关，更大的模型考虑使用更高的 dropout）既有效抑制了过拟合又没有欠拟合。

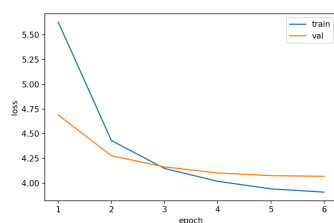


图 21: 7-dropout-0.05

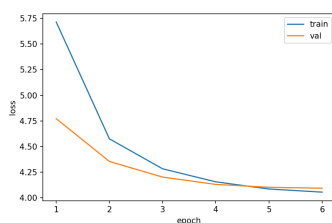


图 22: 7-dropout-0.10

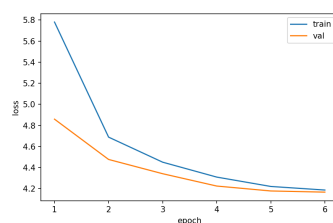


图 23: 7-dropout-0.15

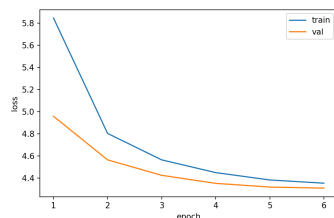


图 24: 7-dropout-0.20

附一：写词尝试

4.6 v0 版本

我们基于对于五言、七言、词的划分，进行古诗词的生成，但是发现但在实际测试中发现，模型生成词时往往存在内容过短、结构不完整的问题。相比五言诗和七言诗，词的生成质量明显不稳定，常常只输出一两句，难以形成完整的词作。

思考之后我们认为这与词这一体裁的特殊性密切相关，因为词并不像五言、七言那样句式相对统一。不同词牌之间在句数、字数、停顿、平仄和整体篇幅上差异较大。如果把所有词牌混合在一起训练，模型很难学习到稳定的格式规律，最终就容易出现生成长度不固定、结构混乱、提前结束等问题。基于词我们进行词牌划分训练。

4.7 v1 版本

我们尝试对词牌进行划分训练，希望让模型在更明确的格式约束下学习词的生成规律，但在很多词牌的样本数量非常有限单独训练时容易出现过拟合，模型难以真正学习到稳定的语言和格律特征。比如如梦令词牌的数据集只有 150 多首。

基于这个问题我们必须重新制定数据集划分策略，选取样本量较充足的词牌单独建模，将

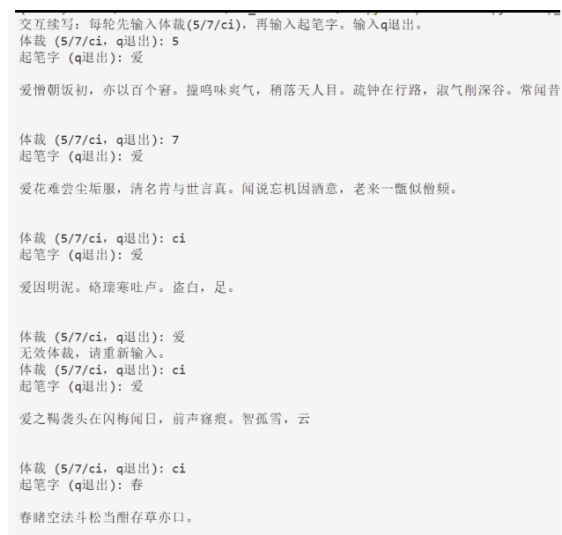


图 25: v0

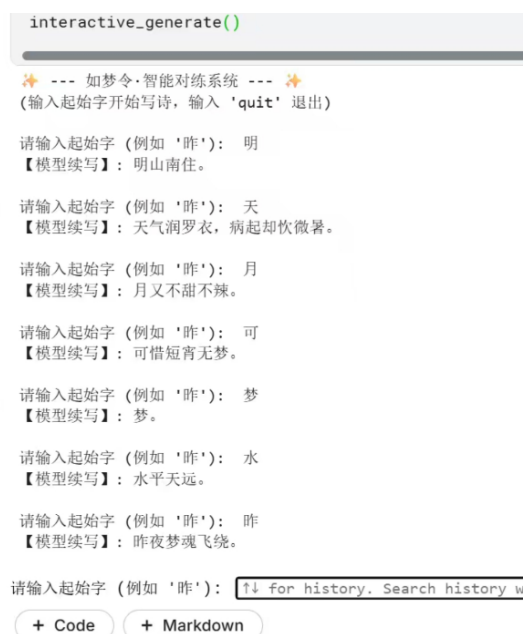


图 26: v1

样本量不足的词牌合并为“其他词牌”类别。最终，我们选择了六组样本量大于 350 的词牌作为主要训练类别，包括：浣溪沙、水调歌头、鹧鸪天、菩萨蛮、蝶恋花，以及“其他词牌”。通过这种方式，既保留了高频词牌相对明确的格式特征，也避免了低频词牌因样本过少而无法训练的问题。

4.8 v2 版本

生成内容不再像早期版本那样过短，整体篇幅更完整，词句之间的连贯性和风格一致性也有所改善。尤其是在选择具体词牌后，模型能够在一定程度上体现出该词牌的句式特点，生成结果的可读性和完整度都有所提高。尤其是在选择具体词牌后，模型能够在一定程度上体现出该词牌的句式特点，生成结果的可读性和完整度都有所提高。

4.9 反思：数据有限且结构复杂

不过，从最终结果分析来看，由于古诗词数据本身数量有限，尤其是按词牌进一步划分后，每一类可用于训练的样本仍然不算充足。因此，模型虽然能够学习到一定的词牌风格和语言表



图 27: v2-1



图 28: v2-2



图 29: v2-3



图 30: v2-4

达，但对于严格的词牌格式、句长规律和平仄格律的掌握仍然不够稳定。换句话说，当前模型已经从“能生成词”提升到了“能按词牌生成相对完整的词”，但距离严格符合词牌格律的高质量生成仍有进一步优化空间。

总体来看，这一阶段的优化过程体现了我们对问题认识的逐步深入：从最初发现词生成过短，到意识到词牌格律差异是核心原因，再到尝试按词牌划分训练，最后根据样本量限制采用“高频词牌单独训练、低频词牌合并”的折中方案。虽然最终结果仍受限于数据规模，但该过程有效提升了模型生成质量，也为后续引入更强的格式约束、扩充词牌数据集、结合规则校验等优化方向奠定了基础。

附二：分工

基础代码	丁佳怡			
调参	何馨怡 (dropout)	何琪铭 (d_model, n_head, n_layer, d_ff)	丁佳怡 (lr, batch_size)	李纳 (block_size, weight decay)
版本迭代	丁佳怡 (平仄, 五言七言, 数据分类清洗, 加速, 重复惩罚)	何馨怡 (押韵, 主题分类, 动态温度)	李纳 (词牌名分类, 数据补充)	何琪铭 (动态温度、打分)
PPT制作	何馨怡 (参数分析)	丁佳怡 (版本迭代)	何琪铭 (项目架构)	李纳 (总结反思)
可视化 (前端、绘图)	何琪铭			
汇报	何馨怡			

图 31: 分工